

◆ What is a System?

A system is a group of interconnected parts or components that work together in an organized way to perform a task or achieve a specific goal.

In simple words, a system is not just a single thing, but a combination of many things working together to do something useful.

◆ Explanation in Simple Terms:

- Think of your body. It is a system.
- Your body has different parts: heart, lungs, brain, stomach, etc.
- Each part does a different job, but they all work together to keep you alive.
- If one part stops working, it affects the whole system.
- That's how a system works – each part has a role, and together they achieve a common purpose.

◆ Formal Definition of a System:

A system is a set of interrelated components or elements that are organized to achieve a specific objective by accepting inputs, processing them, and producing outputs.

◆ Key Features of a System:

1. **Components** – Different parts that make up the system.
2. **Interdependence** – The components depend on each other.
3. **Interaction** – The components communicate or exchange data/materials.
4. **Goal/Purpose** – The system is designed to do something useful.
5. **Input and Output** – Systems take something in (input), do something to it (process), and give something out (output).
6. **Processing** – The transformation that happens inside the system.
7. **Feedback** – Information returned to improve or control the system.

◆ Examples of Systems:

1. Computer System

- Components: CPU, RAM, Keyboard, Monitor, Software
- Purpose: To perform calculations, run programs, and manage data.

2. Banking System

- Components: Customers, Accounts, ATMs, Servers, Staff
- Purpose: To manage money, deposits, withdrawals, and provide financial services.

3. School System

- Components: Students, Teachers, Classrooms, Books, Exams
- Purpose: To provide education and evaluate student learning.

♦ Why Understanding Systems is Important?

- Helps in understanding how things work together in daily life and technology.
- Useful in building software, managing businesses, and solving real-world problems.
- Builds logical thinking, problem-solving skills, and helps in designing better information systems.

♦ System Flow Example:



Input: Raw student data



Processing: Calculate marks, grades, attendance



Output: Final report card

Types of Systems

A system can be classified in different ways depending on how it is built, how it works, how it interacts with the environment, and what it is used for.

Let's understand each type of system with examples and a clear explanation.

♦ 1. Physical System

- These are **tangible systems** — we can see them and touch them.
- They are made up of physical components like machines, tools, and hardware.
- These systems are **mechanical**, **electrical**, or **electronic** in nature.



Example:

- A **car** is a physical system made up of tires, engine, gears, and fuel system.
- A **computer** is also a physical system with components like CPU, monitor, keyboard, and hard drive.



Why Important?

They help us perform physical tasks like moving, computing, or communicating.

♦ 2. Abstract System

- These are **non-physical systems** – you **cannot touch or see them physically**.
- They exist in the **form of ideas, concepts, or rules**.
- Used mostly in **mathematics, logic, and software**.

 Example:

- A **mathematical equation** like $A + B = C$ is an abstract system.
- **Software algorithms** and programming logic are also abstract systems.

 Why Important?

Abstract systems help in planning, calculating, organizing thoughts, and developing logic-based solutions.

♦ **3. Open System**

- These systems **interact with their environment**.
- They take **input from outside**, process it, and give **output back** to the environment.
- Open systems are affected by external conditions like weather, users, or policies.

 Example:

- A **business organization** is an open system – it takes raw materials, processes them, and delivers products to customers.
- A **school** is also an open system – it receives students (input), provides education (process), and creates educated individuals (output).

 Why Important?

Open systems are dynamic and flexible. They can adapt to changes in the environment.

♦ **4. Closed System**

- These systems do **not interact with the environment**.
- They are **self-contained**, isolated, and do not depend on external input/output.
- No influence from outside is allowed.

 Example:

- A **watch mechanism** (manual one) is a closed system. It runs based on internal gears and does not change based on external environment.
- A **basic thermostat** that operates on a fixed rule without feedback is another example. This

 Why Important?

Closed systems are stable but not adaptable. They are useful in controlled environments.

♦ 5. Man-Made System

- These systems are **created by humans** for a specific purpose.
- They can be physical (like machines) or abstract (like programs or organizations).
- Designed for **convenience, productivity, or problem-solving**.

Example:

- A **computer system, train system, or banking system** are all man-made systems.
- **Government systems, hospital systems, or education systems** are also man-made.

Why Important?

Man-made systems help improve life quality, save time, and reduce effort in complex tasks.

♦ 6. Natural System

- These systems occur **naturally in the environment**.
- They are not created by humans.
- They follow the **laws of nature**.

Example:

- The **solar system** is a natural system made of the sun, planets, and moons.
- The **human body** is also a natural system with organs working together.

Why Important?

Natural systems are complex and balanced. Understanding them helps in science, medicine, and environmental studies.

♦ 7. Deterministic System

- These systems work in a **predictable manner**.
- If we know the input, we can always predict the output.
- The outcome is **fixed and known**.

Example:

- A **calculator** gives the same answer every time you press $2 + 2$.
- A **traffic light system** that changes lights based on fixed timing.

Why Important?

They are reliable, suitable for routine or repeated tasks.

♦ 8. Probabilistic System

- These systems work with **uncertainty or chance**.
- Even with the same input, the output may vary.
- The outcome is based on **probability, not fixed rules**.

Example:

- A **weather prediction system** gives chances of rain based on data.
- A **stock market system** reacts to many unpredictable factors.

Why Important?

They help in decision-making where certainty is not possible.

♦ 9. Social System

- These are systems involving **people**, their **roles**, and their **interactions**.
- They are **non-technical systems**, but often supported by technology.

Example:

- A **family system**, **educational system**, or **political system** are social systems.
- They are structured by rules, relationships, and responsibilities.

Why Important?

Social systems organize human life and ensure cooperation and order.

♦ 10. Information System

- An information system is designed to **collect, store, process**, and **deliver** information.
- These systems support decision-making, coordination, control, and analysis.

Example

- A **library management system**, **student database system**, or **online ticket booking system**.

Why Important?

They are essential in almost every business and organization to manage data and operations.

Characteristics of a System

A system has some common features that help it function properly. These features make it easier to understand, design, and manage systems.

Let's look at the key characteristics of any system:

♦ 1. Components

- A system is made up of several **individual parts** or **elements**, called components.
- Each component has a specific role or function within the system.
- All components must work **together properly** for the system to be successful.

 *Example:* In a computer system, components include the CPU, RAM, keyboard, monitor, and software. All these work together.

♦ 2. Interrelationship

- The components in a system are **connected and interact** with each other.
- One component's performance can **affect other components**.
- These relationships are necessary for smooth operation.

 *Example:* In a car, the engine, fuel system, and wheels are connected. If the engine fails, the car won't move.

♦ 3. Organization

- Every system has a **well-defined structure or arrangement**.
- Components are **arranged in a logical and orderly manner** to achieve the goal of the system.

 *Example:* In a school, students, teachers, classrooms, and schedules are organized to provide education.

♦ 4. Purpose or Goal

- Every system exists to achieve a **specific goal or purpose**.
- The goal is the reason why the system was created in the first place.
- All components and processes work toward this goal.

 *Example:* The goal of a banking system is to manage customer accounts and financial transactions.

♦ 5. Input and Output

- A system takes **input** (data, material, or energy), **processes it**, and gives an **output** (result).
- The output is useful and meaningful, based on what the system is meant to do.

 *Example:* A washing machine takes clothes and water as input, washes them (processing), and gives clean clothes as output.

♦ 6. Processing (Throughput)

- The input is passed through a **processing unit** where transformation happens.
- This process is also known as **throughput**.

 *Example:* In a computer, input from the keyboard is processed by the CPU to display something on the screen.

♦ 7. Feedback Mechanism

- A good system can use **feedback** to monitor and improve its performance.
- Feedback is the information the system gets after producing output, which helps it correct errors or improve next time.

 *Example:* A thermostat in an air conditioner receives feedback about room temperature and adjusts the cooling accordingly.

♦ 8. Boundaries

- Every system has a **boundary** that separates it from its environment.
- The boundary defines **what is inside** the system and **what is outside**.
- It helps to focus on the system's components only.

 *Example:* In a company, the employees and internal processes are inside the system, and customers or suppliers are outside the boundary.

♦ 9. Environment

- A system does not exist alone; it operates within an **external environment**.
- This environment can **affect or influence** how the system works.
- The system may also send outputs back to the environment.

 *Example:* A school system is affected by government education policies, public feedback, and social changes.

♦ 10. Subsystems

- A system may be made up of **smaller systems**, called **subsystems**.
- Each subsystem performs its own job, and together they help the main system function.

 *Example:* In a computer system, the input unit, memory unit, and processing unit are all subsystems.

Elements of a System

A system is made up of different **basic elements** that work together to perform its functions and achieve its goal. These elements help in understanding how a system works from start to finish.

♦ 1. Input

- Input refers to the **data, material, or energy** that enters the system from the environment.
- It is the **starting point** of any system operation.
- The quality of output depends on the quality of input.

Example:

In a student result system, student marks and personal information are inputs.

♦ 2. Process (or Throughput)

- This is the **core activity** of the system that converts **input into output**.
- Processing involves actions, calculations, changes, or transformations.
- It is where the actual work happens inside the system.

Example:

In an online shopping system, the process includes order checking, payment verification, and preparing items for delivery.

♦ 3. Output

- Output is the **final result or product** produced by the system after processing the input.
- The output must be **useful, relevant**, and **meet the goal** of the system.
- Outputs may be data, reports, services, decisions, or physical products.

Example:

In a payroll system, the salary slip generated is the output.

♦ 4. Control

- Control is responsible for **monitoring and guiding** the system's operations.
- It ensures that the system is working according to its goal and **doesn't go off track**.
- If something goes wrong, control helps correct it.

Example:

In a vehicle, the brake system is a control element. It helps manage speed and direction.

♦ 5. Feedback

- Feedback is **information about the output** that is sent back into the system.
- It helps the system **adjust** or **improve** its future operations.
- Feedback can be **positive (continue as is)** or **negative (make changes)**.

Example:

In a customer support system, if users complain about long waiting times, that feedback helps in making changes to improve service.



Need for System Development

A **system** is a combination of different components that work together to achieve a goal. Over time, old systems can become **inefficient, outdated, or unable to meet new requirements**, which creates a **need for system development**.

1. To Replace Inefficient Systems

- Old systems may be **slow, manual, or error-prone**.
- Developing a new system can **automate tasks** and **save time**.

Example:

Replacing manual attendance registers with an **online attendance system**.

2. To Meet Changing Business Needs

- Organizations grow and require **new features**.
- A system developed today may need updates tomorrow to **handle more users or services**.

Example:

A shop shifting from offline sales to an **e-commerce website**.

3. To Improve Accuracy and Reduce Errors

- Manual processes are prone to mistakes.
- A new system **reduces human error** with **automated calculations and validations**.

Example:

Online exam systems automatically **calculate marks** without mistakes.

4. To Increase Efficiency and Speed

- A well-developed system **completes tasks faster** and **handles more data**.
- Saves **time, cost, and resources**.

Example:

Banking systems process **online transactions instantly** instead of manual updates.

5. To Provide Better Decision-Making Support

- Modern systems **store, analyze, and report data** to help management take better decisions.

Example:

A school management system shows **attendance trends and results** for quick decisions.

6. To Stay Competitive and Modern

- Businesses must adopt **latest technologies** to survive in the market.
- System development ensures **compatibility with new devices, apps, and platforms**.

Example:

Migrating to **mobile-friendly apps** for customer use.



Role of a System Analyst

A **System Analyst** is a **key person** in the process of developing a system.

They act as a **bridge** between the **users (who need the system)** and the **technical team (who develops the system)**.

Their main role is to **understand the problem, design the solution, and ensure the system meets user needs**.

1. Understanding User Requirements

- The analyst **talks to users** to understand what they need from the system.
- Finds out **problems in the current system** and **improvements required**.

Example:

A college wants a system to manage **attendance, exams, and results**.

2. Studying the Existing System

- The analyst **analyzes the current workflow** to identify **weaknesses and gaps**.
- Prepares a **report** on how the system works and where it needs changes.

Example:

Checking how **attendance is recorded manually** and why it causes **delays or errors**.

3. Preparing System Requirements

- After collecting information, the analyst creates a **Requirement Specification Document**.
- This document **guides developers** to build the right system.

Example:

Listing features like **automatic attendance calculation** or **SMS alerts to parents**.

4. Designing the System

- The analyst **plans how the system will work**.
- Creates **diagrams, flowcharts, and database designs** for the development team.

Example:

Designing the **dashboard layout** for a student portal.

5. Coordinating with Developers and Users

- Explains requirements to **programmers and testers**.
- **Takes feedback** from users to ensure the system is being developed correctly.

Example:

Showing a **prototype** of the attendance module to teachers for approval.

6. Ensuring Quality and Efficiency

- The analyst checks if the system is:
 - **User-friendly**
 - **Error-free**
 - **Efficient** in handling data
- Helps in **testing and verifying** the system before implementation.

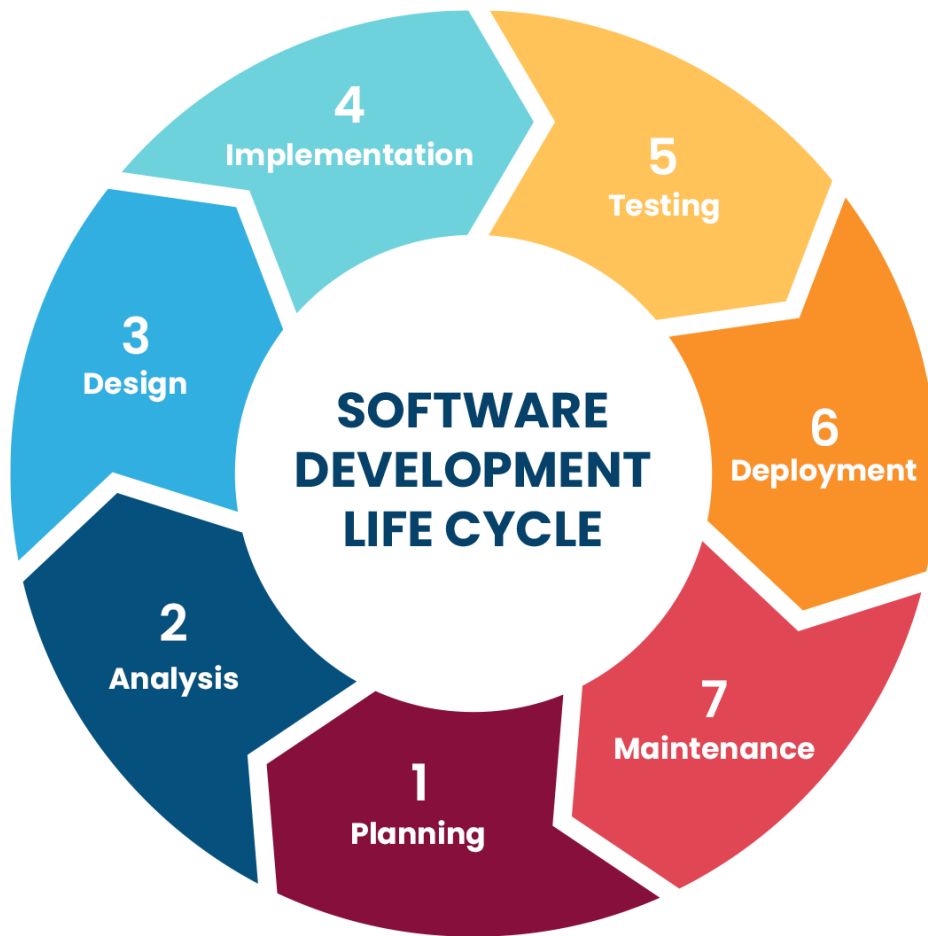
7. Providing Support and Maintenance

- After the system is implemented, the analyst **monitors performance**.
- Suggests **updates and improvements** when needed.

System Development Life Cycle (SDLC)

♦ What is SDLC?

- SDLC stands for **System Development Life Cycle**.
- It is a **step-by-step process** used to **plan, create, test, and deploy** an information system.
- SDLC ensures that systems are developed in a **structured, efficient, and quality-controlled** way.
- It helps avoid mistakes, reduce costs, and deliver a system that meets user needs.



✓ Importance of SDLC

- Provides a **clear roadmap** for system development.
- Ensures that the system is built with **proper planning and testing**.
- Helps in **managing time, resources, and cost** effectively.
- Involves **users, analysts, designers, developers, and testers**.

🔄 Phases of SDLC (7 Main Stages)

♦ 1. Planning Phase

- This is the **first and most important phase**.
- It involves **understanding the problem**, setting goals, and deciding whether a new system is needed.
- Project teams are formed, and **resources, cost, and time** are estimated.

 *Example:* A company wants to build an online booking system. In this phase, they decide what the system will do and how much it will cost.

♦ 2. System Analysis

- In this phase, the analyst collects detailed information from users to understand **what they want**.
- The current system (if any) is studied, and **problems and requirements** are clearly defined.
- The goal is to understand **"what the system should do."**

 *Example:* Interviewing users to know how they book tickets and what problems they face.

♦ 3. System Design

- This phase involves **designing the structure** of the new system.
- It includes creating:
 - Input and output design
 - Data flow diagrams (DFD)
 - User interfaces
 - Database design
- This phase answers the question: **"How will the system work?"**

 *Example:* Designing the login screen, search form, and database for an online ticket booking system.

♦ 4. Development (Coding)

- Programmers now **write the actual code** based on the design created in the previous phase.
- This is the **technical phase** where the system is built using a programming language.
- Developers follow **standards and best practices** to ensure quality.

 *Example:* Using HTML, CSS, and PHP to create the web pages of the ticket system.

♦ 5. Testing

- After development, the system is **tested for errors, bugs, and performance**.
- There are different types of testing:
 - **Unit Testing** – testing individual modules
 - **System Testing** – checking the entire system

- **User Acceptance Testing (UAT)** – testing by users
- The goal is to ensure the system is **error-free and meets user requirements**.

 *Example:* Checking whether the booking page calculates ticket prices correctly and confirms bookings.

◆ 6. Implementation (Deployment)

- Once testing is successful, the system is **installed or delivered** to the users.
- Data from the old system (if any) is transferred.
- Users are trained on how to use the new system.
- Can be done in phases or all at once (known as direct cutover).

 *Example:* Launching the ticket booking system for public use and training the customer support team.

◆ 7. Maintenance

- After the system goes live, it may face **new problems, user requests, or need updates**.
- In this phase, the system is **monitored, maintained, and improved** as needed.
- Maintenance includes fixing bugs, updating features, or adapting to new needs.

 *Example:* Adding a new payment method to the booking system based on user feedback.

✓ Summary: SDLC Phases in Short

1. **Planning** – Decide what to build and if it's possible.
2. **Analysis** – Understand what users need.
3. **Design** – Plan how the system will work.
4. **Development** – Build the actual system (coding).
5. **Testing** – Make sure it works properly.
6. **Implementation** – Install and train users.
7. **Maintenance** – Fix and update as needed.

🧠 Benefits of SDLC:

- Helps build **high-quality systems**.
- Reduces the chance of **project failure**.
- Saves **time and money** by planning ahead.

- Makes sure users get what they really need.

◆ Point of Comparison	🧩 System Analysis	🔧 System Design
1. Definition	Study of existing system and user requirements.	Planning the structure and functioning of the new system.
2. Purpose	Understand what the system should do.	Decide how the system will work.
3. Focus	Focuses on problems, requirements, and user needs.	Focuses on solutions, system structure, and components.
4. Activities Involved	- Requirement gathering- System study- Identifying issues	- Designing input/output screens- Database design- Process planning
5. Output	System Requirement Specification (SRS) document.	System Design Document (including DFDs, ER diagrams, screen layouts).
6. Tools Used	Interviews, questionnaires, DFDs, requirement report	Flowcharts, mockups, ER diagrams, database schemas.
7. Performed By	System Analyst.	System Designer, Developer or Technical Architect.

◆ Point of Comparison	🧩 System Analysis	🔧 System Design
8. Position in SDLC	Comes after planning and before design.	Comes after analysis and before development (coding).
9. Goal	Identify what users want and what the system needs.	Build a blueprint for developers to create the system.
10. Example	Understanding what features teachers want in a result system.	Designing login forms, mark entry forms, and the result display layout.

📘 Need for Structured Analysis and Design

When we develop a system, it is often **complex** and involves many components like **data, processes, and users**.

To **manage this complexity**, we use **Structured Analysis and Design**.

Structured Analysis and Design is a **step-by-step, organized approach** that helps in **planning, analyzing, and designing a system** in a **clear and systematic way**.

Why We Need Structured Analysis and Design

1. Handles Complexity

- Large systems are **difficult to understand as a whole**.
- Structured methods **break the system into smaller, manageable modules**.

Example:

A college system is divided into **admission, attendance, exam, and result modules**.

2. Clear Understanding of Requirements

- Ensures we **fully understand the problem** before creating a solution.
- Uses **diagrams like Data Flow Diagrams (DFDs) to visualize system flow**.

Example:

Showing **how student data moves** from admission → attendance → results.

3. Reduces Errors

- Step-by-step documentation and clear design **reduce mistakes during development**.
- Errors can be **caught early** in the analysis phase.

Example:

Identifying **missing features** before coding starts.

4. Improves Communication

- Structured methods create **visual diagrams and documents**.
- Makes it **easy for users, analysts, and developers to understand the system**.

Example:

Teachers can **review DFDs or flowcharts** to confirm if the system meets their needs.

5. Easy to Maintain and Update

- A structured system is **well-documented** and **organized**, so changes are easier.
- Future upgrades can be done **without affecting the entire system**.

Example:

Adding an **online fee payment module** without disturbing the attendance module.

6. Saves Time and Cost

- Early planning avoids **rework and major errors** later.
- Structured design leads to **faster development and fewer bugs**.

◆ **SSADM Methodology (Structured Systems Analysis and Design Method)**

SSADM stands for Structured Systems Analysis and Design Method. It is a system development methodology widely used in software engineering and information system development. It provides a structured approach to analyze, design, and implement information systems step by step.

1. Introduction

- SSADM was developed in the 1980s in the UK for government IT projects.
- It focuses on analyzing requirements first and then designing a solution using structured techniques like Data Flow Diagrams (DFD), Entity-Relationship Diagrams (ERD), and Logical Data Models.
- It is process-oriented, meaning it studies how data moves and how processes work.

◆ **Main Objectives of SSADM:**

1. **Understand** what the system needs to do.
2. **Analyze** current problems in the system.
3. **Design** a logical and efficient solution.
4. **Document** everything properly to avoid confusion.

◆ **Key Stages of SSADM:**

Stage	Name	Explanation
1	Feasibility Study	Decide if the new system is possible and worth doing (cost, time, benefits).
2	Requirements Analysis	Understand and list what users want the system to do.
3	Requirements Specification	Turn the user needs into a detailed, written technical document.
4	Logical System Specification	Design how the system will work without choosing technology (what it does, not how).
5	Physical Design	Choose tools (software/hardware), design databases, and screens for implementation.

4. Tools and Techniques Used in SSADM

- **Data Flow Diagrams (DFD)** – Show how data moves in the system.
- **Entity Relationship Diagrams (ERD)** – Show data entities and their relationships.
- **Data Dictionary** – Contains definitions of all data items.
- **Logical Data Models** – Represent the structure of data logically.
- **Physical Data Models** – Show how data will be stored in the database.

5. Advantages of SSADM

1. Structured and systematic approach reduces errors.
2. Improves communication between technical teams and users.
3. Well-documented at every stage.
4. Easy to maintain and update in the future.
5. Reduces development risk by clear planning and analysis.

6. Disadvantages of SSADM

1. Time-consuming due to detailed documentation.
2. Expensive for small projects.
3. Rigid and less flexible if requirements change frequently.
4. Complex for small or simple systems.

♦ Real-Life Example:

Imagine building a **school management system**:

- You first study the school's current way of managing data (feasibility).
- Then gather needs (like attendance tracking, fee management).
- Create diagrams showing how data flows between departments (like admin, teachers).
- Design the screens and databases for storing student info.

♦ System Prototype Method (Prototyping Methodology)

The **System Prototype Method** is a software/system development approach where a **working model (prototype)** of the system is created **early in the process** to better understand requirements and design.

♦ Explanation in Simple Language:

Imagine you are making a **mobile app**.

Instead of building the whole app directly, you first create a **rough version** that shows how the app might look and behave.

You show this to users, get feedback, make improvements, and **slowly refine** it until it becomes the final product.

◆ **Key Steps in the Prototyping Method:**

Step	Explanation
1	Requirement Gathering – Talk to users to understand what the need.
2	Build Initial Prototype – Create a simple working model (like a mock-up).
3	User Evaluation – Show the prototype to users and get feedback.
4	Refinement – Improve the prototype based on feedback.
5	Final Implementation – Convert the improved prototype into the final system.

◆ **Types of Prototypes:**

- **Throwaway (Rapid)** – Quickly made, not part of final system, just for understanding.
- **Evolutionary** – Improved step-by-step until it becomes the final system.
- **Incremental** – Built in small parts (modules) and integrated gradually.
- **Extreme Prototyping** (Web apps) – Used mainly for web-based systems.

◆ **Advantages of Prototyping:**

- Users get a **clear view** of how the system will work.
- Easy to detect and fix **problems early**.
- Helps improve **communication** between users and developers.
- Reduces the risk of **building the wrong system**.

♦ Disadvantages:

- Can lead to **incomplete analysis** if too focused on appearance.
- **Frequent changes** can increase cost and time.
- Sometimes users **expect the prototype to be final**, which can cause confusion.

♦ Real-Life Example:

Suppose you're designing an **online food ordering app**:

You build a prototype showing the home page, menu, and order button.

You show it to restaurant owners and customers.

They suggest changes like adding filters, delivery tracking, etc.

You update the prototype, and finally develop the full system based on it.